

# Tarea10

January 28, 2026

## 1 Escuela Politécnica Nacional

### 1.1 [Tarea 10] Ejercicios Unidad 04-C | Descomposición LU

1.1.1 Nombre: Luis Lema

1.1.2 Fecha: 28/01/2025

1.1.3 Curso: GR1CC

1.1.4 Repositorio:

[https://github.com/LuisALema/Metodos\\_Numericos\\_2025B/tree/main/Deberes/Tarea10](https://github.com/LuisALema/Metodos_Numericos_2025B/tree/main/Deberes/Tarea10)

#### Conjunto de Ejercicios

1. Realice las siguientes multiplicaciones matriz-matriz:

a.  $\begin{bmatrix} 2 & -3 \\ 3 & -1 \end{bmatrix} \begin{bmatrix} 1 & 5 \\ 2 & 0 \end{bmatrix}$

b.  $\begin{bmatrix} 2 & -3 \\ 3 & -1 \end{bmatrix} \begin{bmatrix} 1 & 5 & -4 \\ -3 & 2 & 0 \end{bmatrix}$

c.  $\begin{bmatrix} 2 & -3 & 1 \\ 4 & 3 & 0 \\ 5 & 2 & -4 \end{bmatrix} \begin{bmatrix} 0 & 1 & -2 \\ 1 & 0 & -1 \\ 2 & 3 & -2 \end{bmatrix}$

d.  $\begin{bmatrix} 2 & 1 & 2 \\ -2 & 3 & 0 \\ 2 & -1 & 3 \end{bmatrix} \begin{bmatrix} 1 & -2 \\ -4 & 1 \\ 0 & 2 \end{bmatrix}$

```
[1]: import numpy as np

def multiplicar_matrices(matriz_a, matriz_b):
    # Convertir a arrays numpy si no lo son ya
    a = np.array(matriz_a)
    b = np.array(matriz_b)

    # Verificar que las dimensiones son compatibles
    if a.shape[1] != b.shape[0]:
        raise ValueError("Las dimensiones de las matrices no son compatibles_
        ↪para multiplicación")

    return np.matmul(a, b)
```

### Literal a

```
[2]: # Definir matrices
matriz_a = [[2, -3], [3, -1]]
matriz_b = [[1, 5], [2, 0]]

# Calcular resultado
resultado = multiplicar_matrices(matriz_a, matriz_b)
print("Resultado literal a:")
print(resultado)
```

Resultado literal a:

```
[[ -4 10]
 [  1 15]]
```

### Literal b

```
[3]: # Definir matrices
matriz_a = [[2, -3], [3, -1]]
matriz_b = [[1, 5, -4], [-3, 2, 0]]

# Calcular resultado
resultado = multiplicar_matrices(matriz_a, matriz_b)
print("\nResultado literal b:")
print(resultado)
```

Resultado literal b:

```
[[ 11   4  -8]
 [  6  13 -12]]
```

### Literal c

```
[4]: # Definir matrices
matriz_a = [[2, -3, 1], [4, 3, 0], [5, 2, -4]]
matriz_b = [[0, 1, -2], [1, 0, -1], [2, 3, -2]]

# Calcular resultado
resultado = multiplicar_matrices(matriz_a, matriz_b)
print("\nResultado literal c:")
print(resultado)
```

Resultado literal c:

```
[[ -1   5  -3]
 [  3   4 -11]
 [ -6  -7  -4]]
```

### Literal d

```
[5]: # Definir matrices
matriz_a = [[2, 1, 2], [-2, 3, 0], [2, -1, 3]]
matriz_b = [[1, -2], [-4, 1], [0, 2]]

# Calcular resultado
resultado = multiplicar_matrices(matriz_a, matriz_b)
print("\nResultado literal d:")
print(resultado)
```

Resultado literal d:

```
[[ -2   1]
 [-14   7]
 [  6   1]]
```

2. Determine cuáles de las siguientes matrices son no singulares y calcule la inversa de esas matrices:

a. 
$$\begin{bmatrix} 4 & 2 & 6 \\ 3 & 0 & 7 \\ -2 & -1 & -3 \end{bmatrix}$$

c. 
$$\begin{bmatrix} 1 & 1 & -1 & 1 \\ 1 & 2 & -4 & -2 \\ 2 & 1 & 1 & 5 \\ -1 & 0 & -2 & -4 \end{bmatrix}$$

b. 
$$\begin{bmatrix} 1 & 2 & 0 \\ 2 & 1 & -1 \\ 3 & 1 & 1 \end{bmatrix}$$

d. 
$$\begin{bmatrix} 4 & 0 & 0 & 0 \\ 6 & 7 & 0 & 0 \\ 9 & 11 & 1 & 0 \\ 5 & 4 & 1 & 1 \end{bmatrix}$$

```
[6]: def es_no_singular_y_inversa(matriz):
    # Convertir a array numpy si no lo es ya
    m = np.array(matriz)

    # Calcular el determinante
    det = np.linalg.det(m)

    # Verificar si es no singular (determinante != 0)
    if not np.isclose(det, 0):
        inversa = np.linalg.inv(m)
        return True, inversa
    else:
        return False, None
```

Literal a

```
[7]: # Definir matriz
matriz = [[4, 2, 6], [3, 0, 7], [-2, -1, -3]]
```

```
# Verificar y calcular inversa
es_no_singular, inversa = es_no_singular_y_inversa(matriz)
print("Literal a:")
print(f"¿Es no singular?: {'Sí' if es_no_singular else 'No'}")
if es_no_singular:
    print("Matriz inversa:")
    print(inversa)
```

Literal a:

¿Es no singular?: No

Literal b

```
[8]: # Definir matriz
matriz = [[1, 2, 0], [2, 1, -1], [3, 1, 1]]

# Verificar y calcular inversa
es_no_singular, inversa = es_no_singular_y_inversa(matriz)
print("\nLiteral b:")
print(f"¿Es no singular?: {'Sí' if es_no_singular else 'No'}")
if es_no_singular:
    print("Matriz inversa:")
    print(inversa)
```

Literal b:

¿Es no singular?: Sí

Matriz inversa:

```
[[ -0.25   0.25   0.25 ]
 [  0.625 -0.125 -0.125]
 [  0.125 -0.625  0.375]]
```

Literal c

```
[9]: # Definir matriz
matriz = [[1, 1, -1, 1], [1, 2, -4, -2], [2, 1, 1, 5], [-1, 0, -2, -4]]

# Verificar y calcular inversa
es_no_singular, inversa = es_no_singular_y_inversa(matriz)
print("\nLiteral c:")
print(f"¿Es no singular?: {'Sí' if es_no_singular else 'No'}")
if es_no_singular:
    print("Matriz inversa:")
    print(inversa)
```

Literal c:

¿Es no singular?: No

Literal d

```
[10]: # Definir matriz
matriz = [[4, 0, 0, 0], [6, 7, 0, 0], [9, 11, 1, 0], [5, 4, 1, 1]]

# Verificar y calcular inversa
es_no_singular, inversa = es_no_singular_y_inversa(matriz)
print("\nLiteral d:")
print(f"¿Es no singular?: {'Sí' if es_no_singular else 'No'}")
if es_no_singular:
    print("Matriz inversa:")
    print(inversa)
```

Literal d:

¿Es no singular?: Sí

Matriz inversa:

```
[[ 2.50000000e-01  6.16790569e-18  0.00000000e+00  0.00000000e+00]
 [-2.14285714e-01  1.42857143e-01 -0.00000000e+00 -0.00000000e+00]
 [ 1.07142857e-01 -1.57142857e+00  1.00000000e+00 -0.00000000e+00]
 [-5.00000000e-01  1.00000000e+00 -1.00000000e+00  1.00000000e+00]]
```

3. Resuelva los sistemas lineales 4 x 4 que tienen la misma matriz de coeficientes:

$$\begin{array}{ll}
 x_1 - x_2 + 2x_3 - x_4 = 6, & x_1 - x_2 + 2x_3 - x_4 = 1, \\
 x_1 - x_3 + x_4 = 4, & x_1 - x_3 + x_4 = 1, \\
 2x_1 + x_2 + 3x_3 - 4x_4 = -2, & 2x_1 + x_2 + 3x_3 - 4x_4 = 2, \\
 -x_2 + x_3 - x_4 = 5; & -x_2 + x_3 - x_4 = -1.
 \end{array}$$

```
[11]: def resolver_sistemas_lineales(matriz_coef, vectores_terminos):
    # Convertir a arrays numpy
    A = np.array(matriz_coef)
    soluciones = []

    # Verificar si la matriz es cuadrada y no singular
    if A.shape[0] != A.shape[1]:
        raise ValueError("La matriz de coeficientes debe ser cuadrada")

    if np.isclose(np.linalg.det(A), 0):
        raise ValueError("La matriz de coeficientes es singular (no
        invertible)")

    # Resolver cada sistema
    for b in vectores_terminos:
        b_vec = np.array(b)
        solucion = np.linalg.solve(A, b_vec)
        soluciones.append(solucion)
```

```
return soluciones
```

```
[12]: # Matriz de coeficientes (común a ambos sistemas)
matriz_coef = [
    [1, -1, 2, -1],
    [1, 0, -1, 1],
    [2, 1, 3, -4],
    [0, -1, 1, -1]
]

# Vectores de términos independientes
vector_b1 = [6, 4, -2, 5] # Primer sistema
vector_b2 = [1, 1, 2, -1] # Segundo sistema

# Resolver ambos sistemas
try:
    soluciones = resolver_sistemas_lineales(matriz_coef, [vector_b1, vector_b2])

    print("Solución para el primer sistema (b = [6, 4, -2, 5]):")
    print(f"x1 = {soluciones[0][0]:.2f}, x2 = {soluciones[0][1]:.2f}, x3 = _
↪{soluciones[0][2]:.2f}, x4 = {soluciones[0][3]:.2f}")

    print("\nSolución para el segundo sistema (b = [1, 1, 2, -1]):")
    print(f"x1 = {soluciones[1][0]:.2f}, x2 = {soluciones[1][1]:.2f}, x3 = _
↪{soluciones[1][2]:.2f}, x4 = {soluciones[1][3]:.2f}")

except ValueError as e:
    print(f"Error: {str(e)}")
```

Solución para el primer sistema (b = [6, 4, -2, 5]):  
x1 = 3.00, x2 = -6.00, x3 = -2.00, x4 = -1.00

Solución para el segundo sistema (b = [1, 1, 2, -1]):  
x1 = 1.00, x2 = 1.00, x3 = 1.00, x4 = 1.00

4. Encuentre los valores de A que hacen que la siguiente matriz sea singular.

$$A = \begin{bmatrix} 1 & -1 & \alpha \\ 2 & 2 & 1 \\ 0 & \alpha & -\frac{3}{2} \end{bmatrix}.$$

```
[13]: def encontrar_valores_singulares(matriz, parametro=' '):
    try:
        from sympy import symbols, Matrix, solve
```

```

except ImportError:
    raise ImportError("Se requiere la biblioteca sympy para este cálculo.↳
↳Instálela con: pip install sympy")

# Crear símbolo para el parámetro
alpha = symbols(parametro)

# Convertir la matriz a una matriz simbólica
A = Matrix(matriz(alpha))

# Calcular el determinante
det = A.det()

# Resolver det(A) = 0
valores = solve(det, alpha)

return valores

```

```

[14]: # Definir la matriz en función de
def matriz_ejemplo(alpha):
    return [
        [1, -1, alpha],
        [2, 2, 1],
        [0, alpha, -3/2]
    ]

# Encontrar valores que hacen singular la matriz
try:
    valores_singulares = encontrar_valores_singulares(matriz_ejemplo)
    print("Los valores de que hacen singular la matriz son:")
    for val in valores_singulares:
        print(f" = {val.evalf():.4f}") # Mostrar con 4 decimales

except Exception as e:
    print(f"Error: {str(e)}")

```

Los valores de que hacen singular la matriz son:

- = -1.5000
- = 2.0000

5. Resuelva los siguientes sistemas lineales:

$$\text{a.} \quad \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 2 & 3 & -1 \\ 0 & -2 & 1 \\ 0 & 0 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 2 \\ -1 \\ 1 \end{bmatrix}$$

$$\text{b.} \quad \begin{bmatrix} 2 & 0 & 0 \\ -1 & 1 & 0 \\ 3 & 2 & -1 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} -1 \\ 3 \\ 0 \end{bmatrix}$$

```
[15]: def resolver_sistema_factorizado(L, U, b):
    # Convertir a arrays numpy
    L = np.array(L, dtype=float)
    U = np.array(U, dtype=float)
    b = np.array(b, dtype=float)

    # Resolver Ly = b (sustitución hacia adelante)
    y = np.zeros_like(b)
    for i in range(len(b)):
        y[i] = b[i] - np.dot(L[i, :i], y[:i])
        y[i] /= L[i, i]

    # Resolver Ux = y (sustitución hacia atrás)
    x = np.zeros_like(y)
    for i in range(len(y)-1, -1, -1):
        x[i] = y[i] - np.dot(U[i, i+1:], x[i+1:])
        x[i] /= U[i, i]

    return x
```

**Literal a**

```
[16]: # Definir matrices y vector
L = [
    [1, 0, 0],
    [2, 1, 0],
    [-1, 0, 1]
]

U = [
    [2, 3, -1],
    [0, -2, 1],
    [0, 0, 3]
]

b = [2, -1, 1]
```



```
# Resolver sistema
solucion = resolver_sistema_factorizado(L, U, b)
print("Solución literal a:")
print(f"x1 = {solucion[0]:.4f}, x2 = {solucion[1]:.4f}, x3 = {solucion[2]:.4f}")
```

Solución literal a:

x1 = -3.0000, x2 = 3.0000, x3 = 1.0000

**Literal b**

```
[17]: # Definir matrices y vector
L = [
    [2, 0, 0],
    [-1, 1, 0],
    [3, 2, -1]
]

U = [
    [1, 1, 1],
    [0, 1, 2],
    [0, 0, 1]
]

b = [-1, 3, 0]

# Resolver sistema
solucion = resolver_sistema_factorizado(L, U, b)
print("\nSolución literal b:")
print(f"x1 = {solucion[0]:.4f}, x2 = {solucion[1]:.4f}, x3 = {solucion[2]:.4f}")
```

Solución literal b:

x1 = 0.5000, x2 = -4.5000, x3 = 3.5000

**6. Factorice las siguientes matrices en la descomposición LU mediante el algoritmo de factorización LU con  $l_{ii} = 1$  para todas las  $i$ .**

a. 
$$\begin{bmatrix} 2 & -1 & 1 \\ 3 & 3 & 9 \\ 3 & 3 & 5 \end{bmatrix}$$

c. 
$$\begin{bmatrix} 2 & 0 & 0 & 0 \\ 1 & 1.5 & 0 & 0 \\ 0 & -3 & 0.5 & 0 \\ 2 & -2 & 1 & 1 \end{bmatrix}$$

b. 
$$\begin{bmatrix} 1.012 & -2.132 & 3.104 \\ -2.132 & 4.096 & -7.013 \\ 3.104 & -7.013 & 0.014 \end{bmatrix}$$

d. 
$$\begin{bmatrix} 2.1756 & 4.0231 & -2.1732 & 5.1967 \\ -4.0231 & 6.0000 & 0 & 1.1973 \\ -1.0000 & -5.2107 & 1.1111 & 0 \\ 6.0235 & 7.0000 & 0 & -4.1561 \end{bmatrix}$$

```
[18]: def factorizacion_lu_doolittle(A):

    A = np.array(A, dtype=float)
    n = A.shape[0]
    L = np.eye(n) # Matriz identidad (diagonal = 1)
    U = np.zeros((n, n))

    for k in range(n):
        # Calcular U
        for j in range(k, n):
            U[k,j] = A[k,j] - np.dot(L[k,:k], U[:k,j])

        # Calcular L
        for i in range(k+1, n):
            L[i,k] = (A[i,k] - np.dot(L[i,:k], U[:k,k])) / U[k,k]

    return L, U
```

#### Literal a

```
[19]: # Definir matriz
A = [
    [2, -1, 1],
    [3, 3, 9],
    [3, 3, 5]
]

# Factorizar
L, U = factorizacion_lu_doolittle(A)

print("Factorización literal a:")
print("\nMatriz L:")
print(L)
print("\nMatriz U:")
print(U)
```

Factorización literal a:

Matriz L:

```
[[1.  0.  0. ]
 [1.5 1.  0. ]
 [1.5 1.  1. ]]
```

Matriz U:

```
[[ 2. -1.  1. ]
 [ 0.  4.5  7.5]
 [ 0.  0. -4. ]]
```

#### Literal b

```
[20]: # Definir matriz
A = [
    [1.012, -2.132, 3.104],
    [-2.132, 4.096, -7.013],
    [3.104, -7.013, 0.014]
]

# Factorizar
L, U = factorizacion_lu_doolittle(A)

print("\nFactorización literal b:")
print("\nMatriz L:")
print(L)
print("\nMatriz U:")
print(U)
```

Factorización literal b:

Matriz L:

```
[[ 1.          0.          0.          ]
 [-2.10671937  1.          0.          ]
 [ 3.06719368  1.19775553  1.          ]]
```

Matriz U:

```
[[ 1.012      -2.132      3.104      ]
 [ 0.         -0.39552569 -0.47374308]
 [ 0.          0.         -8.93914077]]
```

**Literal c**

```
[21]: # Definir matriz
A = [
    [2, 0, 0, 0],
    [1, 1.5, 0, 0],
    [0, -3, 0.5, 0],
    [2, -2, 1, 1]
]

# Factorizar
L, U = factorizacion_lu_doolittle(A)

print("\nFactorización literal c:")
print("\nMatriz L:")
print(L)
print("\nMatriz U:")
print(U)
```

Factorización literal c:

Matriz L:

```
[[ 1.      0.      0.      0.      ]
 [ 0.5     1.      0.      0.      ]
 [ 0.     -2.      1.      0.      ]
 [ 1.     -1.33333333 2.      1.     ]]
```

Matriz U:

```
[[2.  0.  0.  0. ]
 [0.  1.5 0.  0. ]
 [0.  0.  0.5 0. ]
 [0.  0.  0.  1. ]]
```

Literal d

```
[22]: # Definir matriz
A = [
    [2.1756, 4.0231, -2.1732, 5.1967],
    [-4.0231, 6.0000, 0, 1.1973],
    [-1.0000, -5.2107, 1.1111, 0],
    [6.0235, 7.0000, 0, -4.1561]
]

# Factorizar
try:
    L, U = factorizacion_lu_doolittle(A)
    print("\nFactorización literal d:")
    print("\nMatriz L:")
    print(L)
    print("\nMatriz U:")
    print(U)
except np.linalg.LinAlgError as e:
    print(f"\nError en literal d: {str(e)}")
```

Factorización literal d:

Matriz L:

```
[[ 1.      0.      0.      0.      ]
 [-1.84919103  1.      0.      0.      ]
 [-0.45964332 -0.25012194  1.      0.      ]
 [ 2.76866152 -0.30794361 -5.35228302  1.     ]]
```

Matriz U:

```
[[ 2.1756     4.0231    -2.1732     5.1967     ]
 [ 0.         13.43948042 -4.01866194 10.80699101]
 [ 0.          0.        -0.89295239  5.09169403]
 [ 0.          0.          0.         12.03612803]]
```

7. Modifique el algoritmo de eliminación gaussiana de tal forma que se pueda utilizar para resolver un sistema lineal usando la descomposición LU y, a continuación, resuelva los siguientes sistemas lineales.

- a.  $2x_1 - x_2 + x_3 = -1,$   
 $3x_1 + 3x_2 + 9x_3 = 0,$   
 $3x_1 + 3x_2 + 5x_3 = 4.$
- b.  $1.012x_1 - 2.132x_2 + 3.104x_3 = 1.984,$   
 $-2.132x_1 + 4.096x_2 - 7.013x_3 = -5.049,$   
 $3.104x_1 - 7.013x_2 + 0.014x_3 = -3.895.$
- c.  $2x_1 = 3,$   
 $x_1 + 1.5x_2 = 4.5,$   
 $-3x_2 + 0.5x_3 = -6.6,$   
 $2x_1 - 2x_2 + x_3 + x_4 = 0.8.$
- d.  $2.1756x_1 + 4.0231x_2 - 2.1732x_3 + 5.1967x_4 = 17.102,$   
 $-4.0231x_1 + 6.0000x_2 + 1.1973x_4 = -6.1593,$   
 $-1.0000x_1 - 5.2107x_2 + 1.1111x_3 = 3.0004,$   
 $6.0235x_1 + 7.0000x_2 - 4.1561x_4 = 0.0000.$

```
[23]: def resolver_sistema_lu(A, b):
    A = np.array(A, dtype=float)
    b = np.array(b, dtype=float)
    n = A.shape[0]

    # Inicializar L y U
    L = np.eye(n)
    U = A.copy()

    # Eliminación gaussiana con pivoteo parcial
    for k in range(n-1):
        # Pivoteo parcial
        max_row = np.argmax(np.abs(U[k:, k])) + k
        U[[k, max_row]] = U[[max_row, k]]
        b[[k, max_row]] = b[[max_row, k]]
        L[[k, max_row], :k] = L[[max_row, k], :k]

        # Eliminación
        for i in range(k+1, n):
            L[i, k] = U[i, k] / U[k, k]
            U[i, k:] -= L[i, k] * U[k, k:]

    # Sustitución hacia adelante (Ly = b)
    y = np.zeros(n)
    for i in range(n):
        y[i] = b[i] - np.dot(L[i, :i], y[:i])
```

```

# Sustitución hacia atrás ( $Ux = y$ )
x = np.zeros(n)
for i in range(n-1, -1, -1):
    x[i] = (y[i] - np.dot(U[i, i+1:], x[i+1:])) / U[i, i]

return x, L, U

```

### Literal a

```

[24]: # Definir sistema
A = [
    [2, -1, 1],
    [3, 3, 9],
    [3, 3, 5]
]
b = [-1, 0, 4]

# Resolver
x, L, U = resolver_sistema_lu(A, b)
print("Solución literal a:")
print(f"x1 = {x[0]:.6f}, x2 = {x[1]:.6f}, x3 = {x[2]:.6f}")
print("\nMatriz L:")
print(L)
print("\nMatriz U:")
print(U)

```

Solución literal a:

x1 = 1.000000, x2 = 2.000000, x3 = -1.000000

Matriz L:

```

[[ 1.         0.         0.         ]
 [ 0.66666667  1.         0.         ]
 [ 1.         -0.         1.         ]]

```

Matriz U:

```

[[ 3.  3.  9.]
 [ 0. -3. -5.]
 [ 0.  0. -4.]]

```

### Literal b

```

[25]: # Definir sistema
A = [
    [1.012, -2.132, 3.104],
    [-2.132, 4.096, -7.013],
    [3.104, -7.013, 0.014]
]
b = [1.984, -5.049, -3.895]

```

```

# Resolver
try:
    x, L, U = resolver_sistema_lu(A, b)
    print("\nSolución literal b:")
    print(f"x1 = {x[0]:.6f}, x2 = {x[1]:.6f}, x3 = {x[2]:.6f}")
    print("\nMatriz L:")
    print(L)
    print("\nMatriz U:")
    print(U)
except np.linalg.LinAlgError as e:
    print(f"\nError en literal b: {str(e)}")

```

Solución literal b:

x1 = 1.000000, x2 = 1.000000, x3 = 1.000000

Matriz L:

```

[[ 1.          0.          0.          ]
 [-0.68685567  1.          0.          ]
 [ 0.32603093 -0.21424728  1.          ]]

```

Matriz U:

```

[[ 3.104      -7.013      0.014      ]
 [ 0.         -0.72091881 -7.00338402]
 [ 0.          0.         1.59897957]]

```

**Literal c**

```

[26]: # Definir sistema
A = [
    [2, 0, 0, 0],
    [1, 1.5, 0, 0],
    [0, -3, 0.5, 0],
    [2, -2, 1, 1]
]
b = [3, 4.5, -6.6, 0.8]

# Resolver
x, L, U = resolver_sistema_lu(A, b)
print("\nSolución literal c:")
print(f"x1 = {x[0]:.6f}, x2 = {x[1]:.6f}, x3 = {x[2]:.6f}, x4 = {x[3]:.6f}")
print("\nMatriz L:")
print(L)
print("\nMatriz U:")
print(U)

```

Solución literal c:

x1 = 1.500000, x2 = 2.000000, x3 = -1.200000, x4 = 3.000000

Matriz L:

```
[[ 1.          0.          0.          0.          ]
 [ 0.          1.          0.          0.          ]
 [ 1.          0.66666667  1.          0.          ]
 [ 0.5         -0.5         0.375       1.          ]]
```

Matriz U:

```
[[ 2.          0.          0.          0.          ]
 [ 0.          -3.         0.5         0.          ]
 [ 0.          0.          0.66666667  1.          ]
 [ 0.          0.          0.          -0.375       ]]
```

**Literal d**

```
[27]: # Definir sistema
A = [
    [2.1756, 4.0231, -2.1732, 5.1967],
    [-4.0231, 6.0000, 0, 1.1973],
    [-1.0000, -5.2107, 1.1111, 0],
    [6.0235, 7.0000, 0, -4.1561]
]
b = [17.102, -6.1593, 3.0004, 0.0000]

# Resolver
try:
    x, L, U = resolver_sistema_lu(A, b)
    print("\nSolución literal d:")
    print(f"x1 = {x[0]:.6f}, x2 = {x[1]:.6f}, x3 = {x[2]:.6f}, x4 = {x[3]:.6f}")
    print("\nMatriz L:")
    print(L)
    print("\nMatriz U:")
    print(U)
except np.linalg.LinAlgError as e:
    print(f"\nError en literal d: {str(e)}")
```

Solución literal d:

x1 = 2.939851, x2 = 0.070678, x3 = 5.677735, x4 = 4.379812

Matriz L:

```
[[ 1.          0.          0.          0.          ]
 [-0.66790072  1.          0.          0.          ]
 [ 0.36118536  0.14002434  1.          0.          ]
 [-0.16601644 -0.37924771 -0.5112737   1.          ]]
```

Matriz U:

```
[[ 6.0235       7.          0.          -4.1561      ]
 [ 0.          10.67530506  0.          -1.57856219 ]]
```



```
[ 0.      0.     -2.1732    6.91885959]
[ 0.      0.      0.      2.24878393]]
```

```
[ ]:
```